

# Reviewer Pack — Følner-Defect Floor Forces Logarithmic Communication Overhea...

Entry 723804a64233 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 12:42:37 UTC

## Følner-Defect Floor Forces Logarithmic Communication Overhead on Equality-Lifted Gadgets

Entry ID: 723804a64233 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 12:42:37 UTC

### 1. Conjecture

**Field A** (mathematical branch): Coarse Geometric Lifting (CGL) — coarse geometry / geometric group theory applied to communication complexity **Field B** (complexity object): Deterministic two-party communication complexity  $CC(EQ_k \circ G^n)$  of the lifted Equality function on  $n$  copies of a fixed MetricGadget  $G$

#### Statement:

Let  $G = (X, Y, g, d)$  be a MetricGadget with  $|X|=|Y|=k$  and Hamming-graph metric  $d$ , and let  $EQ_k : \{0,1\}^n \rightarrow \{0,1\}$  denote Equality on  $n$  bits with  $Q(EQ_k) = n$ . Define the Følner defect of the lifted space at scale  $R$  as  $\delta_R(n) = \min \text{ over finite } \bar{F} \subset (X \times Y)^n \text{ of } |\partial_R F|/|F|$ , where  $\partial_R F$  is the  $d \oplus$ - $R$ -boundary. Then for every gadget  $G$  satisfying axiom A4 (Property A) and every  $n$ ,  $CC(EQ_k \circ G^n) \geq Q(EQ_k) \cdot \lceil \log_2(1/\delta_R(n) + 1) \rceil$  for  $R = \text{diam}(G)$ , and equivalently  $CLC(EQ_k, G)$  is bounded below by  $n \cdot \log_2(\text{AsdimCertify}(\text{LiftedInputSpace}_n, \text{diam}(G)) + 1)$ . In particular, when  $\text{AsdimCertify}$  returns multiplicity  $m$  at scale  $R$ , every deterministic protocol must use at least  $n \cdot \log_2(m+1)$  bits.

#### Rationale (proposer's reasoning):

This sub-conjecture directly tests Axiom A2 (Protocol  $\rightarrow$  Cover) combined with Axiom A4 (Property-A Transfer). A2 says any cost- $c$  protocol yields a RoeCover of multiplicity  $\leq 2^c$ , so  $\log_2$  of the minimum cover multiplicity lower-bounds  $CC$ . A4 promotes that bound by the query complexity factor  $Q(f)$ . For Equality,  $Q(EQ_k) = n$  and any non-trivial RoeCover at scale  $\text{diam}(G)$  of the  $n$ -fold lift must have multiplicity at least  $k^{\Omega(1)}$  per coordinate by amenability obstructions (Følner defect), giving a per-coordinate logarithmic multiplier. The conjecture should hold because the Følner-defect quantity  $\delta_R(n)$  shrinks geometrically in  $n$  for product metrics, forcing the cover multiplicity — and hence the protocol cost — to scale linearly with  $n$  with a logarithmic-in- $k$  coefficient.

**Taxonomy category:** LIFTING (status at proposal time: )

**Framework membership:** framework fw\_6997a27304, role: elaboration

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 62ca14a738aafe01

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

Across all  $(G, n)$  with  $k \in \{2, 3, 4\}$  and  $n \in \{1, 2, 3, 4\}$  (12 configurations, treated as seeds), the inequality  $CC(EQ_k \cdot G^n) \geq n \cdot \log_2(m_n + 1)$  must hold in  $\geq 11/12$  cases, with mean slack  $(CC - n \cdot \log_2(m_n + 1)) \geq 0$  and no configuration violating by more than 0.5 bits.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.86       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.88       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.83       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.92       | SAFE             | SAFE             |

### 4. Novelty audit

**Verdict:** NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - lifted equality communication complexity gadget deterministic lower bound - Property A asymptotic dimension communication complexity lower bound - Følner boundary coarse geometry query-to-communication lifting

**Top relevant hits considered:** - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [http://arxiv.org/abs/cs/9906008v2] A Lower Bound on the Average-Case Complexity of Shellsort - [http://arxiv.org/abs/1904.13056v4] Query-to-Communication Lifting Using Low-Discrepancy Gadgets - [http://arxiv.org/abs/1411.4413v2] Observation of the rare  $B_s^0 \rightarrow \mu^+ \mu^-$  decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R - [http://arxiv.org/abs/1903.06084v8] Lifting coarse homotopies - [http://arxiv.org/abs/2405.06438v2] Building weight-free Følner sets for Yu's Property A in coarse geometry

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90s$  wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from collections import defaultdict

# Implement basic linear algebra operations manually
def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
```

```

        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    M = [A[i] + b[i] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
        M[i], M[max_row] = M[max_row], M[i]
        for j in range(i+1, n):
            factor = M[j][i] / M[i][i]
            M[j] = [M[j][k] - factor * M[i][k] for k in range(n)]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = M[i][-1]
        for j in range(i+1, n):
            x[i] -= M[i][j] * x[j]
        x[i] /= M[i][i]
    return x

def hamming_distance(a, b):
    return sum(1 for i in range(len(a)) if a[i] != b[i])

# Define the MetricGadget class
class MetricGadget:
    def __init__(self, X, Y, g, d):
        self.X = X
        self.Y = Y
        self.g = g
        self.d = d

    def distance(self, x1, y1, x2, y2):
        return self.d(x1, y1) + self.d(x2, y2)

# Implement the Følner defect calculation
def folner_defect(G, R, n):
    X, Y, g, d = G.X, G.Y, G.g, G.d
    k = len(X)
    total_points = k ** (2 * n)
    min_boundary_ratio = float('inf')

    for F in range(total_points):
        boundary_count = 0
        for i in range(n):
            x1, y1 = X[F % k], Y[(F // k) % k]
            x2, y2 = X[(F + 1) % k], Y[((F + 1) // k) % k]
            if d(x1, x2) > R or d(y1, y2) > R:
                boundary_count += 1
        min_boundary_ratio = min(min_boundary_ratio, boundary_count / total_points)

    return min_boundary_ratio

```

```

# Implement the  $CC(EQ_k \circ G^n)$  calculation using rank/partition-bound LP
def cc_eqk_gn(G, n):
    X, Y, g, d = G.X, G.Y, G.g, G.d
    k = len(X)
    m = 2 ** (n * math.log2(k))

    # Construct the communication matrix
    C = [[0 for _ in range(m)] for _ in range(m)]
    for i in range(m):
        x1, y1 = X[i % k], Y[(i // k) % k]
        for j in range(m):
            x2, y2 = X[j % k], Y[(j // k) % k]
            C[i][j] = d(x1, x2) + d(y1, y2)

    # Solve the rank/partition-bound LP
    b = [0 for _ in range(m)]
    A = [[0 for _ in range(m)] for _ in range(m)]
    for i in range(m):
        for j in range(m):
            if i != j:
                A[i][j] = C[i][j]

    x = gaussian_elimination(A, b)
    return sum(x)

# Define the run_trial function
def run_trial(seed: int) -> dict:
    random.seed(seed)
    k_values = [2, 3, 4]
    n_values = [1, 2, 3, 4]

    results = []
    for k in k_values:
        X = [f"x{i}" for i in range(k)]
        Y = [f"y{i}" for i in range(k)]
        g = lambda x, y: hamming_distance(x, y)
        d = lambda a, b: sum(1 for i in range(len(a)) if a[i] != b[i])
        G = MetricGadget(X, Y, g, d)

        for n in n_values:
            R = math.inf # Diameter of the gadget
            m_n = folner_defect(G, R, n)
            cc_eqk_gn_value = cc_eqk_gn(G, n)

            results.append({
                "metric_name": "CC(EQ_k \circ G^n)",
                "metric_value": cc_eqk_gn_value,
                "instances_tested": 1,
                "conjecture_holds": cc_eqk_gn_value >= n * math.log2(m_n + 1),
                "counterexample": ""
            })

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

    return {
        "seed": seed,
        "mean_metric_value": mean_metric_value,
        "support_fraction": support_fraction
    }

# Main block to run trials and print results
if __name__ == "__main__":
    seeds = [11, 23, 37, 53, 71] if len(sys.argv) < 2 else [int(arg) for arg in sys.argv[1:]]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    # Compute mean and support fraction across all seeds
    total_results = []
    for seed in seeds:
        with open(f"trial_{seed}.json", "r") as f:
            trial_result = json.load(f)
            total_results.append(trial_result)

    mean_metric_value = sum(result["mean_metric_value"] for result in total_results) / len(total_results)
    support_fraction = sum(result["support_fraction"] for result in total_results) / len(total_results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = min(total_results, key=lambda x: x["support_fraction"])["seed"]
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d88ab7be.py", line 147, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d88ab7be.py", line 123, in run_trial
    cc_eqk_gn_value = cc_eqk_gn(G, n)
                      ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d88ab7be.py", line 88, in cc_eqk_gn
    C = [[0 for _ in range(m)] for _ in range(m)]
          ~~~~~^
TypeError: 'float' object cannot be interpreted as an integer

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with a `TypeError` before any data was produced (`'float'` object cannot be interpreted as an integer in `cc_eqk_gn` at line 88), so no configurations were evaluated and the pre-registered support criterion cannot be assessed. | next: Cast `m` to `int` (e.g., `m = int(round(m))`) when constructing the cost matrix in `cc_eqk_gn`, then rerun the 12 configurations to obtain CC vs  $n \cdot \log_2(m_{n+1})$  data.

## 11. Audit log (LLM calls)

**Total LLM calls:** 6

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | preregistration | claude_max    | opus             | 0         | 0   | 7593         |
| 2 | novelty         | claude_max    | opus             | 0         | 0   | 7747         |
| 3 | novelty         | claude_max    | opus             | 0         | 0   | 9557         |
| 4 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14060        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17496        |
| 6 | judge           | claude_max    | opus             | 0         | 0   | 5744         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 62197 ms total latency. Provider mix: {'claude\_max': 4, 'ollama\_remote': 2}

(full prompt+response transcripts available in `research/audit/723804a64233.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/723804a64233.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/723804a64233.tar.gz` (if generated)